FlightMood

AI SENTIMENT

Prediction for Airline Comments



Presented By:

Muhammad Nugi Ramadhan

Product Overview

FlightMood is an AI-powered web application designed to analyze and predict the sentiment of airline passenger comments as either positive or negative. By leveraging advanced machine learning algorithms, such as Long Short-Term Memory (LSTM) networks, FlightMood provides accurate sentiment predictions in real-time.



FlightMood is user-friendly, with a simple interface where users can input comments and instantly receive sentiment predictions. With this innovative solution, airlines can elevate their customer experience strategy and maintain a competitive edge in the aviation industry.



Problem & Solution





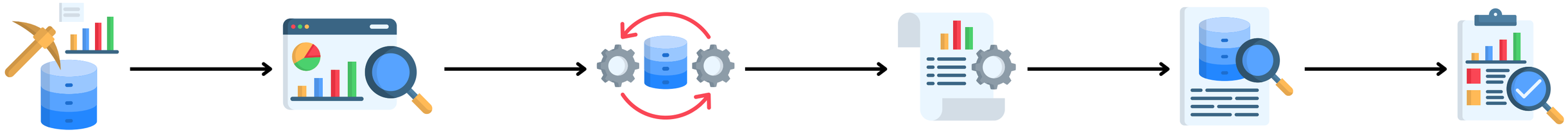
Problem

Airlines face the challenge of manually analyzing thousands of passenger comments every day, a time-consuming process that hinders the ability to gain quick and accurate insights into customer satisfaction. Passengers' dissatisfaction is often undetected in real-time, resulting in a negative customer experience and delayed service improvements.

Solution

FlightMood provides an AI-driven solution with automatic sentiment analysis that quickly and accurately classifies passenger comments as positive or negative. The platform offers real-time insights into customer perceptions, allowing airlines to proactively address issues and enhance the passenger experience. With data-driven decision-making, FlightMood helps airlines optimize services and boost customer satisfaction and loyalty.

Workflow



Representing Data

Exploratory Data Analysis (EDA)

Data Preprocessing

Model Development

Model Evaluation

Prediction Example

Dataset Description

Dataset From Kaggle

This dataset, sourced from Kaggle, consists of 14,640 entries with two primary columns:

- airline_sentiment: The classification of the sentiment expressed in the text (Positive, Negative, or Neutral).
- text: The content of the message reflecting the sentiment regarding the airline experience.



(14640, 2)

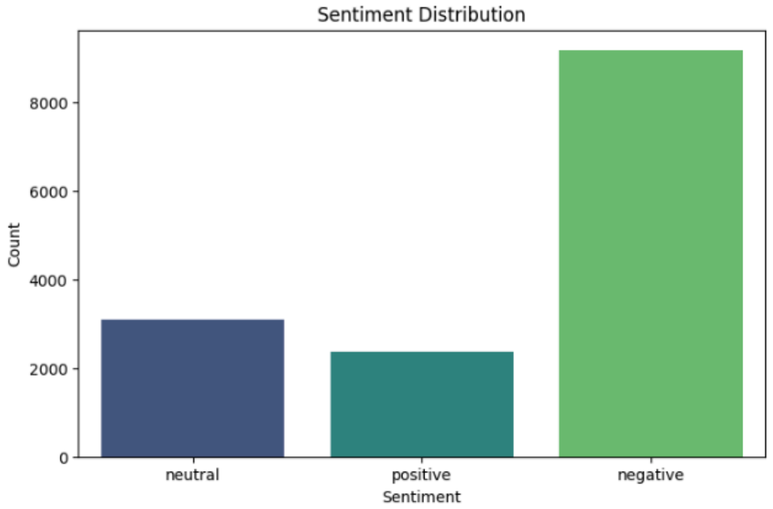
	text	airline_sentiment
0	@VirginAmerica What @dhepburn said.	neutral
1	@VirginAmerica plus you've added commercials t...	positive
2	@VirginAmerica I didn't today... Must mean I n...	neutral
3	@VirginAmerica it's really aggressive to blast...	negative
4	@VirginAmerica and it's a really big bad thing...	negative

Exploratory Data Analysis (EDA)

1. Sentiment Distribution Visualization

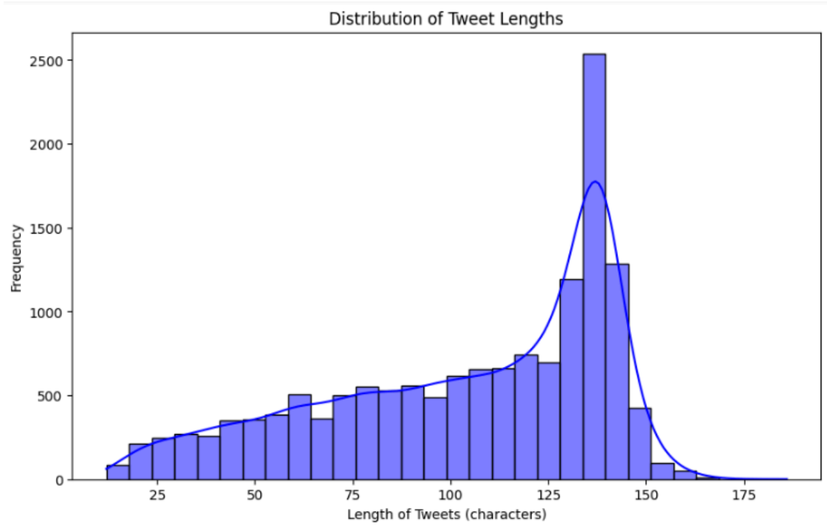
Negative: 9178 comments (62.6%)
Neutral: 3099 comments (21.2%)
Positive: 2363 comments (16.1%)

```
airline_sentiment
negative    9178
neutral     3099
positive    2363
Name: count, dtype: int64
```



2. Length of Tweets Analysis

Most tweets are between 100 and 150 characters in length. This is clear because the histogram has a peak around 130 characters, indicating that this is the most frequent tweet length.



3. Remove Neutral Sentiments

Removing neutral sentiments allows for a more focused analysis of the strongest positive and negative opinions. This eliminates the ambiguity of neutral sentiments and helps to highlight the trends and patterns within the more impactful sentiments.

```
airline_sentiment
negative    9178
positive    2363
Name: count, dtype: int64
```

Data Preprocessing

```
[ ]  # Encode sentiments as binary labels (0 for negative, 1 for positive)
     sentiment_label = review_df.airline_sentiment.factorize()

     # Extract tweet texts
     tweet = review_df.text.values
```

1. Sentiment Encoding

- The code snippet encodes sentiments as binary labels (0 for negative, 1 for positive) using the ***factorize()*** method.
- This step assigns numerical values to the sentiments for further processing.

2. Tokenization and Padding

- Tokenize tweets: Tokenizer tweets to ensure that all tweets have a standardized length.
- Pad sequences: Pad sequences to ensure uniform input length for the model.

```
[ ]  # Tokenize tweets
     tokenizer = Tokenizer(num_words=5000)
     tokenizer.fit_on_texts(tweet)

     # Convert texts to sequences
     encoded_docs = tokenizer.texts_to_sequences(tweet)

     # Pad sequences to ensure uniform input length
     padded_sequence = pad_sequences(encoded_docs, maxlen=200)
```

Model Development

1. Define the LSTM Model and Compile the Model

Developed an LSTM-based model with word embeddings, dropout layers, and a sigmoid activation function for binary sentiment classification. Compiled the model using binary cross-entropy loss and Adam optimizer for training.

```
[ ]  # Define the LSTM model
     embedding_vector_length = 32
     vocab_size = 10000  # Example vocabulary size, adjust if necessary

     model = Sequential()
     model.add(Embedding(vocab_size, embedding_vector_length, input_length=200))
     model.add(SpatialDropout1D(0.25))
     model.add(LSTM(50, dropout=0.5, recurrent_dropout=0.5))
     model.add(Dropout(0.2))
     model.add(Dense(1, activation='sigmoid'))

     # Compile the model
     model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

     # Print model summary
     print(model.summary())
```

```
[ ]  # Train the model
     history = model.fit(padded_sequence, sentiment_label[0], validation_split=0.2, epochs=5, batch_size=32)
```

```
Epoch 1/5
289/289 ━━━━━━━━━━━━━━━━━━━━ 42s 122ms/step - accuracy: 0.7766 - loss: 0.5389 - val_accuracy: 0.8883 - val_loss: 0.2648
Epoch 2/5
289/289 ━━━━━━━━━━━━━━━━━━━━ 39s 116ms/step - accuracy: 0.8772 - loss: 0.3094 - val_accuracy: 0.9216 - val_loss: 0.1936
Epoch 3/5
289/289 ━━━━━━━━━━━━━━━━━━━━ 53s 157ms/step - accuracy: 0.9158 - loss: 0.2260 - val_accuracy: 0.9216 - val_loss: 0.1936
Epoch 4/5
289/289 ━━━━━━━━━━━━━━━━━━━━ 34s 118ms/step - accuracy: 0.9403 - loss: 0.1723 - val_accuracy: 0.9329 - val_loss: 0.1754
Epoch 5/5
289/289 ━━━━━━━━━━━━━━━━━━━━ 45s 130ms/step - accuracy: 0.9496 - loss: 0.1414 - val_accuracy: 0.9281 - val_loss: 0.1865
```

2. Train the Model

Trained the LSTM model over 5 epochs with 80/20 train-validation split, achieving 94.9% training accuracy and 92.8% validation accuracy. Loss consistently decreased across epochs, indicating effective learning.

Model Evaluation

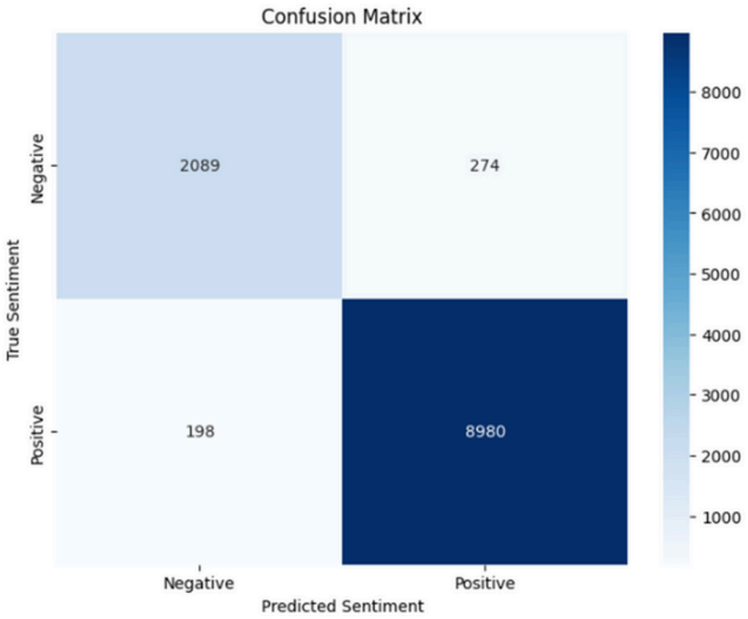
1. Evaluate on Test Data

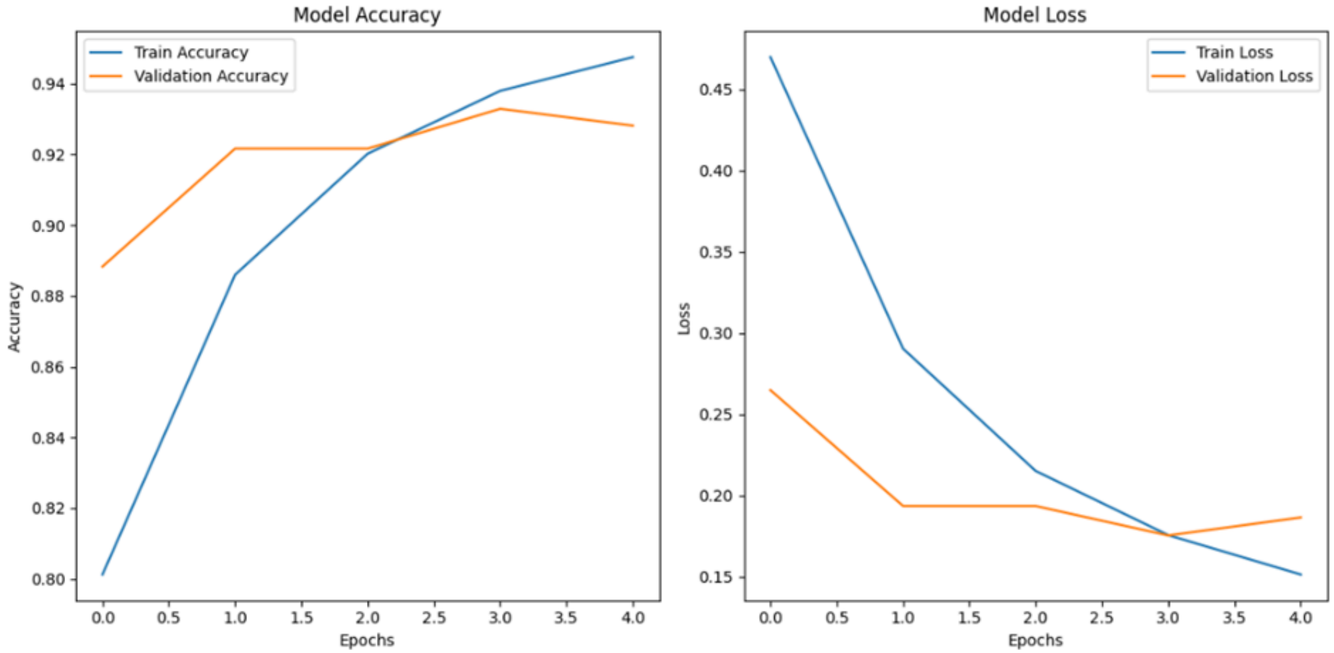
Test Loss: 11,6%
Test Accuracy: 95,9%

```
Test Loss: 0.11570536345243454
Test Accuracy: 0.9591023325920105
```

2. Confusion Matrix

The confusion matrix shows the model's performance with 2,089 correctly predicted negative sentiments and 8,980 correctly predicted positive sentiments, alongside minimal misclassifications. Overall, the model demonstrates high accuracy and reliability.





3. Validation Metrics Visualization

The accuracy and loss graphs illustrate the model's performance over training epochs. Training accuracy consistently improves, reaching 94%, while validation accuracy stabilizes at 92%, indicating strong generalization with minimal overfitting.

Prediction Example

1. Define a Prediction Function

A custom function is defined to predict sentiment by converting input text into sequences, padding it to match the model's input size, and using the trained LSTM model to classify the sentiment.

```
[ ]  def predict_sentiment(text):
         # Convert text to a sequence of numbers
         tw = tokenizer.texts_to_sequences([text])
         # Pad the sequence to ensure consistent length with the training data
         tw = pad_sequences(tw, maxlen=200)
         # Make a prediction with the model
         prediction = int(model.predict(tw).round().item())
         # Display the predicted label
         print("Predicted label: ", sentiment_label[1][prediction])
```

```
[ ]  # Sentence to test the prediction
     test_sentence1 = "The flight was delayed for hours and it was horrible."
     predict_sentiment(test_sentence1)

     1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step
     Predicted label:  negative
```

2. Test Sentences for Prediction

A test sentence is passed to the prediction function, and the model correctly identifies the sentiment as 'negative' based on the input text.

```
[18] # Sentence to test the prediction
     test_sentence2 = "Great flight! The crew was friendly and the service was excellent!"
     predict_sentiment(test_sentence2)

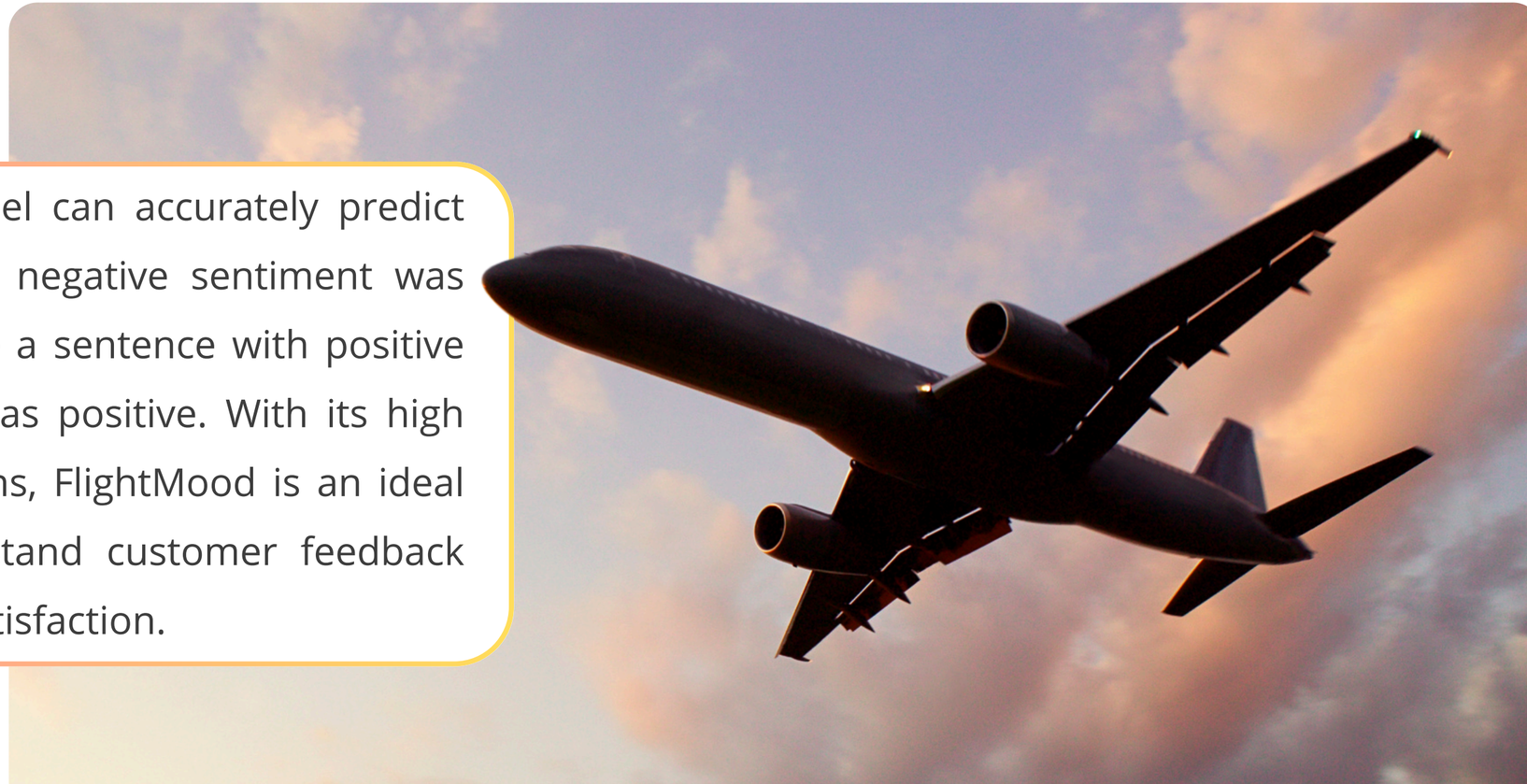
     1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step
     Predicted label:  positive
```

A test sentence with positive sentiment is passed to the model, and it accurately predicts the sentiment as 'positive'.

Conclusion

The LSTM model developed in FlightMood is highly suitable for analyzing airline passenger comments in real-time. With a validation accuracy of 92.8% and a test accuracy of 95.9%, the model has proven to be reliable and capable of generalizing effectively.

Further testing shows that the model can accurately predict sentiment. A sample sentence with negative sentiment was correctly classified as negative, while a sentence with positive sentiment was accurately predicted as positive. With its high performance and accurate predictions, FlightMood is an ideal solution for helping airlines understand customer feedback and effectively enhance passenger satisfaction.



Thank You



Link Portfolio

Link Colab

www.linkedin.com/in/nugi-ramadhan







